Lab 8 — Exercise Answer Key

Lab Exercise Answer Key — Lab 8: Working with Related Data (university schema)

Exercise 1: Advanced Reads

Part A — each course with its enrolled-student count (JOIN + GROUP BY):

```
SELECT c.title, COUNT(e.student_id) AS student_count
FROM courses c
LEFT JOIN enrolments e ON e.course_id = c.id
GROUP BY c.id
ORDER BY student_count DESC;
```

A LEFT JOIN keeps courses with zero enrolments (count 0) — an INNER JOIN would silently drop them.

Part B — students not enrolled in any course (subquery with NOT IN):

```
SELECT name FROM students
WHERE id NOT IN (SELECT student_id FROM enrolments);
```

Part C — the view, and Part D selecting from it for one student:

```
CREATE VIEW enrolment_details AS
  SELECT s.name AS student, c.title AS course, e.grade
  FROM enrolments e
  JOIN students s ON e.student_id = s.id
  JOIN courses  c ON e.course_id  = c.id;

-- Part D: query the view for one student and confirm it matches the join
SELECT * FROM enrolment_details WHERE student = 'Suda Srisawat';
```

This should return exactly the same rows as running the underlying three-table JOIN directly — the view just saves you from retyping it.

Exercise 2: Read Related Data — GET /students/:id/courses

routes/students.routes.js — add the nested route:

```
router.get('/:id/courses', c.courses);
```

controllers/students.controller.js — join enrolments to courses, filtered by the student id:

```
exports.courses = async (req, res) => {
  const [rows] = await pool.execute(`
    SELECT c.id, c.title, e.grade
    FROM enrolments e
    JOIN courses c ON e.course_id = c.id
    WHERE e.student_id = ?`, [req.params.id]);
  res.json(rows);
};
```

Test by visiting /students/1/courses directly in the browser — no tooling needed for a GET route.

Exercise 3: Shape Nested JSON — GET /students/:id

This replaces the plain getOne() lookup with a version that also assembles the student's courses array from the same JOIN pattern used in Exercise 2. A student with no enrolments still gets courses: [], never an error, because the LEFT JOIN keeps the student row even with no matching enrolments.

```
exports.getOne = async (req, res) => {
  const [rows] = await pool.execute(`
    SELECT s.id, s.name, s.major, c.id AS course_id, c.title
    FROM students s
    LEFT JOIN enrolments e ON e.student_id = s.id
    LEFT JOIN courses c ON e.course_id = c.id
    WHERE s.id = ?`, [req.params.id]);

  if (rows.length === 0) return res.status(404).json({ error: 'student not found' });

  const { id, name, major } = rows[0];
  const courses = rows
    .filter(r => r.course_id !== null)
    .map(r => ({ id: r.course_id, title: r.title }));

  res.json({ id, name, major, courses });
};
```

Check the shape in the browser at /students/1 (has courses) and /students/5 (no enrolments → courses: []) using the sample data.

Exercise 4: Create and Remove Links

routes/students.routes.js:

```
router.post('/:id/courses', c.enrol);
router.delete('/:id/courses/:courseId', c.unenrol);
```

controllers/students.controller.js — enrolling inserts a junction row; unenrolling deletes one. Both catch the foreign-key / duplicate-key errors the slides describe, turning them into clear status codes instead of a crash:

```
exports.enrol = async (req, res) => {
  const { course_id, grade } = req.body;
  if (!course_id) return res.status(400).json({ error: 'course_id is required' });
  try {
    const [r] = await pool.execute(
      'INSERT INTO enrolments (student_id, course_id, grade) VALUES (?, ?, ?)',
      [req.params.id, course_id, grade ?? null]);
    res.status(201).json({ id: r.insertId });
  } catch (err) {
    if (err.code === 'ER_NO_REFERENCED_ROW_2')
      return res.status(400).json({ error: 'no such course' });
    if (err.code === 'ER_DUP_ENTRY')
      return res.status(409).json({ error: 'already enrolled in this course' });
    throw err;
  }
};

exports.unenrol = async (req, res) => {
  const [r] = await pool.execute(
    'DELETE FROM enrolments WHERE student_id = ? AND course_id = ?',
    [req.params.id, req.params.courseId]);
  if (r.affectedRows === 0)
    return res.status(404).json({ error: 'enrolment not found' });
  res.status(204).end();
};
```

POST and DELETE can't be triggered from a browser address bar. Test them either with the front-end built in Exercise 5, or with a GUI REST client such as the Thunder Client VS Code extension or Postman — both avoid the quoting problems curl causes on macOS.

Exercise 5: Relationships on the Page

public/index.html additions — a course picker and an enrol form alongside the existing course list:

```
<select id="courseSelect"></select>
<form id="enrolForm"><button type="submit">Enrol</button></form>
```

public/app.js — render the courses array from Exercise 3 with a remove button per row, populate the select from GET /courses, and wire up submit/click handlers that POST/DELETE then reload:

```
async function show(id) {
  const s = await (await fetch(`/students/${id}`)).json();
  name.textContent = s.name;
  courseList.innerHTML = s.courses.map(c =>
    `<li>${c.title}
       <button data-course="${c.id}">remove</button></li>`
  ).join('');
}

async function loadCourseOptions() {
  const courses = await (await fetch('/courses')).json();
  courseSelect.innerHTML = courses
    .map(c => `<option value="${c.id}">${c.title}</option>`).join('');
}

enrolForm.onsubmit = async (e) => {
  e.preventDefault();
  await fetch(`/students/${studentIdInput.value}/courses`, {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ course_id: courseSelect.value })
  });
  show(studentIdInput.value);
};

courseList.addEventListener('click', async (e) => {
  const courseId = e.target.dataset.course;
  if (!courseId) return;
  await fetch(`/students/${studentIdInput.value}/courses/${courseId}`, { method: 'DELETE' });
  show(studentIdInput.value);
});
```

Re-loading the student after every change (as show() does) keeps the page in sync with the database — there's no separate client-side state to get out of sync.